CAFEPULSE

Ecosystem Final Audit & Release Readiness Report

Prepared For: CafePulse Launch Readiness Board

Revision: 6.0 (Ecosystem Stabilization)

Status: GO WITH CONDITIONS

Date: June 5, 2026

CafePulse Final Release Readiness Report

This report evaluates the readiness of the CafePulse desktop software, marketing web assets, and code repositories, providing a final launch score and release recommendation.

1. Launch Scorecard

Evaluation Vector	Score	Status	Audit Justification
Product Readiness	**85 / 100**	**NEAR READY**	Robust core engine (database WAL, validation, logs, vault, discovery, scans). However, the vast Network and Advanced workspace pages contain simulated UI placeholders with mock data.
Commercial Readiness	**90 / 100**	**READY (MANUAL)**	Offline HWID licensing engine is fully implemented with 100 serial keys. Commercial buy buttons route to manual QRIS bank transfer inquiries.
Website Readiness	**95 / 100**	**READY WITH CONDITIONS**	Mapped correctly to the official URL `youbellkey.github.io/cafepulse-site/`. 404, sitemaps, and robots are operational. Fallback release links in `main.js` must be updated to avoid old URL.
Installer Readiness	**90 / 100**	**NEAR READY**	Inno Setup folders and version profiles are designed. A medium-severity background thread shutdown traceback bug must be fixed before compiling binaries.
GitHub Readiness	**92 / 100**	**READY**	Structure is clean with READMEs and license files. Older documentation files contain incorrect URL namespace and need removal or update.

2. Release Status Analysis

A. Ready Features (What actually works)

- **Central Boot & System Logging**: Startup environment check, Safe Mode recovery windows, and global traceback logs writing.
- **Database WAL Engine**: Autocreation of SQLite schema versioning, WAL tracking, and auto-backup restoration.
- **Secure Vault**: Encrypted credentials storage.
- **Network Sweeps**: non-blocking ARP/Ping scanning to find online clients.

- **Voucher Generation Manager**: Random token generator, package data tracking, and dynamic pushing to MikroTik RouterOS API.
- **Central Website Assets**: Responsive HTML pages, Open Graph cards, sitemap, and robots.txt.

B. Not Ready / Placeholders (Simulated features)

- **MikroTik Config Workspaces**: Settings for DNS, Firewall, NAT, PPP, Wireless, Bridge, VLAN, and Queues are high-fidelity UI mockups. Clicking buttons triggers simulator dialog boxes with no backend router execution.
- **Voucher PDF Printing**: Cetak PDF button operates as a layout simulator dialog.
- **Executable Installers**: No installation files or setup binaries have been built.
- **Dynamic Payment Gateway**: Automated checkouts are absent (manual payment QRIS flow mapped).

3. Identified Release Blockers

🔴 Critical Blockers (0 items)

- *None.*

🟠 High Blockers (1 item)

Thread Shutdown Race Traceback

- **Problem**: Background thread workers (like `WiFiWorker` ARP sweeps) accessing the database after it has been closed on shutdown trigger traceback errors.
- **Impact**: Prints `AttributeError` log tracebacks during application shutdown.
- **Resolution**: In `MainWindow.closeEvent()`, call thread termination routines (`worker.quit()`, followed by `worker.wait(5000)`) and block the main loop exit until the worker thread confirms termination.

🟡 Medium Blockers (1 item)

Website Fallback Release Link Error

- **Problem**: The fallback link inside `website/js/main.js` points to `yubelki/cafepulse` instead of the official `youbellkey/cafepulse-site` directory.
- **Resolution**: Update the fallback URL inside `main.js` before public site deployment.

4. Recommended Order of Work

1. **Bugfix Threading Sequence**: Patch `MainWindow.closeEvent()` to wait for active background QThreads cleanly.
2. **Patch Website JS Fallback**: Correct the repository URL fallback in `website/js/main.js`.
3. **Execute Packager Script**: Run `python build.py` to compile the desktop binary packages (`CafePulse_Free.zip` and `CafePulse_Professional.zip`) and clean the legacy Basic/Pro zip files.
4. **Deploy GitHub Pages Web Portal**: Upload the corrected website files to the official GitHub repository.
5. **Compile Inno Setup Installers**: Build the native setup files for Windows distribution.

5. Final Recommendation

Final Launch Recommendation: GO WITH CONDITIONS

The CafePulse core framework, user registration, local database engines, client scans, and voucher generators are stable. The simulated dashboards in the Advanced and Network workspaces are acceptable for a **Version 1.0 (Beta Launch)**, provided that users are informed that advanced MikroTik configurations are currently visual placeholders. Once the High and Medium blockers (thread shutdown traceback and website fallback url link) are patched, the project is fully ready for installer compilation.

CafePulse Product Reality Audit Report

This report evaluates the actual implementation status of all CafePulse software features, distinguishing between fully functional code, partially implemented systems, user interface placeholders, and features that exist solely in documentation.

1. Executive Summary

A critical code-level scan and execution verification was performed across all workspaces, modules, and UI views in the CafePulse repository. The software presents a highly robust architectural core (database management, settings, global logging, credential vault, discovery scanning, and local network sweeps). However, many configuration pages in the Network and Advanced views function as high-fidelity user interface placeholders displaying simulated data.

2. Feature Classification Matrix

Feature / Module	Actual Code Path	Reality Status	Detailed Code and Functional Status
Startup Validation	`main.py` (L65-158)	**FULLY FUNCTIONAL**	Verifies Python version (>= 3.12), folder permissions, config structures, and core packages.
Safe Mode Recovery	`main.py` (L159-198)	**FULLY FUNCTIONAL**	Renders custom dark-themed GUI error box if validation fails. Prevents silent crashes.
Local Database Engine	`core/database/db_manager.py`	**FULLY FUNCTIONAL**	Operates SQLite in WAL mode. Manages automatic migration and corruption recovery.
Secure Credential Vault	`core/security/`	**FULLY FUNCTIONAL**	AES-256 local credentials encryption to protect MikroTik passwords.
Discovery & Neighbor Scan	`core/mikrotik/router_discovery.py`	**FULLY FUNCTIONAL**	Non-blocking multi-threaded probes of TCP 8291, 8728, and 8729 API ports.
Auto-Reconnect Engine	`core/mikrotik/connection_manager.py`	**FULLY FUNCTIONAL**	Exponential backoff logic. Automatic connection recovery when resuming from system sleep.
WiFi Network Scanner	`modes/home_wifi/wifi_scanner.py`	**FULLY FUNCTIONAL**	Performs ARP sweeps and ping scans to discover hosts and resolve vendor MACs.
	`core/iam/`		

Voucher Code Provisioning	`voucher_manager.py`	**FULLY FUNCTIONAL**	Generates unique alphanumeric voucher tokens and writes them directly to MikroTik Hotspot via API.
Voucher Generator (Form & CSV)	`ui/widgets/iam/iam_vouchers.py`	**FULLY FUNCTIONAL**	Bulk generation parameters form, displays table entries, and exports codes to CSV format.
Cetak PDF Voucher Layout	`ui/widgets/iam/iam_vouchers.py` (L243)	**PLACEHOLDER**	Clicking the button displays a simulator info dialog but does not generate or print a vector PDF layout.
Hotspot Active & Servers	`ui/widgets/network/net_hotspot.py`	**PLACEHOLDER**	Table elements load static mock entries (`hotspot1`, `cp-8291`). Dialog simulates HTML login page uploads.
DHCP Leases Manager	`ui/widgets/network/net_ip_dhcp.py`	**PLACEHOLDER**	IP/DHCP tables use static rows. "Make Static" button displays simulated dialog only.
Backup & Restore Manager	`ui/widgets/network/net_backup.py`	**PARTIAL**	Table shows static rows. Backups/Restore buttons set internal flags to test main window close safety behavior.
IP/DNS Configuration	`ui/widgets/network/net_dns.py`	**PLACEHOLDER**	Renders static DNS configuration text fields and static cache table.
Firewall & NAT Manager	`ui/widgets/network/net_firewall.py`	**PLACEHOLDER**	Renders checkable basic toggles and static advanced tables. Rules are not written to router.
Bandwidth Queues (QoS)	`ui/widgets/network/net_queue.py`	**PLACEHOLDER**	Displays static queue trees and interface listings with mock rate-limit info.
VLAN & Routing Manager	`ui/widgets/network/net_routing.py`	**PLACEHOLDER**	Renders static routing paths and VLAN interfaces list.
System Info & Logging	`ui/widgets/network/net_system.py`	**PLACEHOLDER**	Shows static system resources and mock system event logs.

Real-Time Traffic Graphs	`ui/widgets/network/net_traffic.py`	**PLACEHOLDER**	Renders empty layouts / mock curves for bandwidth data.
Access Control (ACL)	`ui/widgets/network/net_access_control.py`	**PLACEHOLDER**	Simulated MAC locking rules.
PPP & Interface Config	`ui/widgets/network/net_ppp.py` / `net_interfaces.py`	**PLACEHOLDER**	Simulated PPPoE sessions lists and physical interface speed sliders.
Wireless (WLAN) Manager	`ui/widgets/network/net_wifi.py`	**PLACEHOLDER**	Renders static SSID text fields and mock signal level gauges.

3. Reality Analysis of Core Workspaces

A. Operations Workspace (IAM)

- **Reality**: The core of Operations—managing local database sync, creating random tokens, and provisioning them onto the router—is **fully implemented**.
- **Placeholders**: The visual printing layout generator (Cetak PDF Voucher) and the live session logs manager are placeholders.

B. Network & Advanced Workspaces (MikroTik Settings)

- **Reality**: The code currently contains high-fidelity visual forms designed to mirror the future features. No connection logic pushes DNS, Firewall, Queues, PPP, Wireless, or VLAN commands to the MikroTik RouterOS API from these pages.
- **Status**: **UI Placeholders** awaiting future Phase 4 and Phase 5 integration.

4. Documentation vs. Reality Gaps

The following features described in the official master roadmap (`docs/MASTER_PRODUCT_RELEASE_ROADMAP.md`) do not have functional implementations in the current source code:

1. **Automated Scheduled Backups**: Roadmap claims automated daily/weekly backup schedules and direct restore wizards. Code only supports static mockup listings with manual simulation flags.
2. **VLAN Creation Wizard**: Roadmap outlines a step-by-step VLAN builder. Code only has a static table with no configuration backend.
3. **Firewall and QoS (Queues) Manager**: Roadmap describes dynamic traffic shaping and firewall toggles. Code is purely visual with simulated settings.
4. **Voucher PDF Printing**: Manual states printable voucher layouts. Code has no rendering library dependencies (like ReportLab or QPrinter canvas mappings) and runs a simulator dialog.

CafePulse Release Architecture Plan

This document outlines the package structure, asset inclusions, legal documentation, and runtime dependencies for the Free and Professional Editions of CafePulse.

1. Product Editions Packaging

To preserve code simplicity, CafePulse uses a single code branch. The edition features are unlocked at runtime based on the license state stored in the SQLite database and settings config.

A. CafePulse Free Edition

- **Scope**: Targeted for single-owner operations, home WiFi testing, and simple network diagnostics.
- **Included Modules**:
  - Home WiFi Mode (dynamic ARP/Ping client discovery sweeps).
  - Startup Validator & Safe Mode Recovery.
  - Local Database Storage (WAL enabled, 30-day traffic prune).
  - System Log & Diagnostics Viewer.
  - Basic Settings (Light/Dark themes).
  - MikroTik Neighbor & Port Discovery (TCP Winbox/API scanning).
- **Limitations**:
  - Voucher batch size limited to 10 tokens.
  - Multi-router saving disabled (only 1 active profile profile).
  - Advanced network management pages display lock widgets.

B. CafePulse Professional Edition

- **Scope**: Designed for commercial cafés, RT/RW Net operators, small hotels, and advanced network administrators.
- **Included Modules**:
  - Unlocked Voucher Batch Generator (up to 500 vouchers per run).
  - Multi-Router Credentials Vault and connection list.
  - Full MikroTik Operations Mode (real-time bandwidth monitor + DHCP stats).
  - AI Network Insights & Traffic Analytics charts.
  - Advanced Network Workspace forms (Firewall, NAT, DNS, VLAN, PPP, QoS, Backup simulators).
  - EULA compliance activation binding (1 License = 1 PC, 5-Year Update Entitlement).

2. Mandatory Distribution Assets

Any compiled distribution package must package the following files and folders:

```
CafePulse/
├── CafePulse.exe           # Main compiled executable
├── assets/
│   ├── icon.ico            # Main window application icon
│   ├── logo.png            # Topbar branding asset
│   └── splash.png          # App boot splash image
├── config/
│   └── settings.json       # Default configuration templates (copied on first boot)
├── docs/
│   └── legal/
│       ├── eula.md         # End User License Agreement
```

```
│       └── 📄 privacy.md      # Data Privacy policies (100% local processing)
└── 📄 LICENSE.txt             # Core software copyright notice
```

3. Required Documentation Bundle

The documentation must reside locally within the release payload as well as on the official web portal:

1. `README_FREE.md`: Guides administrators through setting up client sweeps and standard WiFi network checking.
2. `README_PROFESSIONAL.md`: Provides instruction for connection setup on RouterOS API, credential storage safety, offline license activation key files, and bulk voucher generation.
3. `EULA (legal/eula.md)`: Clarifies the 1 PC node activation lock, Best Effort Support model, and the 5-Year Update Entitlement boundaries.

4. Runtime Dependencies & Verification

The compiled package runs on Windows 10/11 (64-bit) systems. The following packages must be frozen into the executable package by PyInstaller:

Import Name	Package Name (PyPI)	Version	Function in CafePulse
`PyQt6`	`PyQt6`	`6.7.1`	Main desktop application windowing framework.
`pyqtgraph`	`pyqtgraph`	`0.13.7`	High-performance graph canvas for real-time charting.
`mac_vendor_lookup`	`mac-vendor-lookup`	`0.1.12`	Resolves MAC addresses to vendor manufacturers locally.
`cryptography`	`cryptography`	`43.0.1`	Secures the MikroTik password vault using AES-256.
`psutil`	`psutil`	`6.0.0`	Queries host machine system load and loopback sockets.
`routeros_api`	`routeros-api`	`0.21.0`	Establishes API socket connection to RouterOS.

Note: Optional libraries must not trigger compilation failures. If `routeros_api` is absent in developer runtimes, the app gracefully boots into home network sweep modes and displays informative warnings in logs.

CafePulse Installer Readiness Audit Report

This report evaluates the status of the compilation pipeline, runtime recovery features, system dependencies, database schema setups, update behaviors, and uninstall procedures of the CafePulse PyQt6 application.

1. PyInstaller Build System & Reproducibility

A. Code Compilation Audit

- **Script**: `build.py`
- **Compiling Engine**: PyInstaller version `6.10.0`.
- **Build Mechanism**:
    1. Executes PyInstaller in windowed mode (`--windowed`), bundling imports into a folder hierarchy (`--onedir`).
    2. Forces hidden import mapping for `routeros_api` to prevent dynamic load failures.
    3. Uses Pillow (PIL) to dynamically crop and package PNG logos into multi-resolution `icon.ico` executable icons at runtime.
    4. Copies dependencies, assets, and config, then outputs zipped archive binaries into `exports/`.
- **Reproducibility Gaps**:
    - **Finding**: The `exports/` directory currently contains legacy zipped files named `CafePulse_Basic.zip` and `CafePulse_Pro.zip` (compiled in a previous release lifecycle using legacy licensing terms).
    - **Status**: Outdated build. The current build script (`build.py`) is written to output `CafePulse_Free.zip` and `CafePulse_Professional.zip`. Running it will create the correctly named zip files, but the old zip archives must be manually removed to prevent developer/deployment confusion.

2. Desktop Startup Validation & Database Lifecycle

A. Environment Probing (`main.py` L65-158)

- **Python Target Check**: Enforces a strict Python `>= 3.12` validation check at boot.
- **Storage Writable Assessment**: Touch-tests target directories (`logs/`, `exports/`) to verify user execution permissions.
- **Dependency Auditing**: Queries `core/runtime/dependency_registry.py` to identify missing components. Core library failures trigger a Safe Mode boot.
- **Safe Mode Dialog UI**: Built inside `main.py` (L159-198) as a dark-styled PyQt6 box displaying errors clearly instead of throwing tracebacks or silently terminating.

B. SQLite Database Initialization

- **File Location**: Initialized dynamically based on `settings.json` configurations (defaults to `cafepulse.db`).
- **Integrity Validation**: Runs a database schema check (`PRAGMA integrity_check`) at boot.
- **Robustness Mode**: Configures database journal tracking to WAL (Write-Ahead Logging) to prevent data corruption during unexpected power losses or system crashes.
- **Corruption Auto-Recovery**: If SQLite integrity check fails or a DatabaseError is raised, it automatically renames the corrupt file to `cafepulse.db.bak` and initializes a fresh database to keep the app operational.

3. Critical Software Threading Race Bug on Shutdown

During shutdown, the application has a medium-severity race condition:

- **Location**: `MainWindow.closeEvent()` calls `_finalize_and_exit()` inside `ui/windows/main_window.py`.
- **Traceback**: `WiFiWorker error: 'NoneType' object has no attribute 'execute' (AttributeError)`
- **Root Cause**:
    1. When closing, the app triggers `self._stop_all_workers()` which flags `self._running = False` and waits.
    2. However, the background scan loop inside `WiFiWorker.run()` is executing `self._scanner.run_scan()`. This scan utilizes ARP sweeps and pings which can take up to 10+ seconds depending on network latency.
    3. `WiFiWorker.stop()` only waits for 5 seconds (`self.wait(5000)`) before returning.
    4. The main thread continues, calls `db.close()`, and exits, setting the internal SQLite connection pointer `self._conn` to `None`.
    5. The background `WiFiWorker` thread, still running inside `run_scan()`, attempts to write results via `self._db.upsert_device()` and crashes with `AttributeError`.
- **Correction Plan**:
    - Implement interruptible checks inside the scan loops.
    - Ensure `main.py` blocks termination until all QThreads have confirmed shutdown.

4. Updates & Uninstallation Safety

- **Settings Portability**: CafePulse is local-first. User profiles, scan history, settings configs, and secure credential vaults reside inside the application folder (`config/settings.json`, `cafepulse.db`).
- **Update Security**: To prevent configuration loss during application updates, the installer must NOT overwrite `config/settings.json` or `cafepulse.db` if they already exist on the target system.
- **Uninstall Cleanliness**: The uninstaller must cleanly purge files. It should prompt the user: "*Do you want to delete your local database, connection profiles, and configuration logs? (Yes/No)*" to prevent accidental deletion of their network setups.

CafePulse Installer Architecture Review

This report details the architectural design and structural specifications for compiling the CafePulse Free Edition and Professional Edition installers.

1. Selected Installer Engine

For the Windows deployment platform, we recommend **Inno Setup (Version 6+)** for the following technical reasons:

- **Zero Overhead**: Generates single-file, highly compressed setup executables without requiring external runtimes.
- **Silent Installation**: Supports standard CLI flags (`/VERYSILENT`, `/SUPPRESSMSGBOXES`) crucial for deployment by network technicians.
- **Rollback Protection**: Automatically rolls back file states if installation is cancelled or encounters write errors.

2. Target Installation Folder Structure

The setup wizard will extract the application package into the system Program Files folder (using the `{autopf}\CafePulse` constant):

```
{autopf}\CafePulse\
├── CafePulse.exe           # Main compiled PyQt6 desktop binary
├── LICENSE.txt             # Commercial license and EULA terms
├── README_FREE.md          # Guide for Free Edition users
├── README_PROFESSIONAL.md  # Guide for Professional Edition users
├── _internal\              # PyInstaller runtime folder containing DLLs and library files
├── assets\                 # Branding assets, logos, and UI splash graphics
├── config\                 # Default runtime configuration settings
│   └── settings.json       # Initial user preference configurations
├── logs\                   # [Created Writable] Local execution logger folder
└── exports\                # [Created Writable] Local voucher PDF export folder
```

3. Desktop, Start Menu, & Uninstaller Rules

- **Shortcuts**:
  - Create a Desktop shortcut: `{userdesktop}\CafePulse.lnk` pointing to `CafePulse.exe`.
  - Create a Start Menu shortcut folder: `{userprograms}\CafePulse\CafePulse.lnk`.
- **EULA Agreement Flow**:
  - The installer wizard will load `LICENSE.txt` and display it in a scrollable text area. The user **must** select the "*I accept the agreement*" radio button to unlock the "Next" button.
- **Uninstaller Specification**:
  - Creates a clean uninstaller registered in the Windows Add/Remove Programs panel.
  - *Critical Rule*: The uninstaller will remove all extracted binaries, DLLs, and registry keys. However, it **must** prompt the user: "*Apakah Anda ingin menghapus data database lokal (cafepulse.db) dan pengaturan konfigurasi?*" If the user selects "No", the local SQLite database and `settings.json` file inside the user application directory are preserved to prevent accidental data loss.

4. Edition-Specific Activation Flow

To maintain code unified efficiency, both Free and Professional distributions will use a **single, unified codebase** initialized with the following execution profiles:

A. Free Edition Flow

- The user installs the Free package.

- Upon startup, the application detects no `license.lic` validation file inside `config/`.
- The GUI defaults to the Free Workspace layout, disabling the voucher generator, scheduling backups, and blocking multi-router connections.

B. Professional Edition Flow

- The user installs the Professional package.
- Upon first startup, the client presents an activation dialog.
- **Online Activation**: The user inputs their owner name and serial key, and the client creates an encrypted local `license.lic` file bound to their HWID.
- **Offline Activation**:
    1. The user inputs their details and hits "Generate Activation Request".
    2. The client exports a base64-encoded `*.licreq` file.
    3. The user transfers this file to an internet-connected device and emails it to `cafepulse.network@gmail.com`.
    4. The developer returns an encrypted `*.lic` file.
    5. The user clicks "Import Activation File" on their offline machine.
    6. The client decrypts and verifies the HWID hash match, unlocking the Professional Edition.

CafePulse Build System Audit Report

This report evaluates the build pipeline, compiler specifications, and dependency bundling operations configured in `build.py`.

1. PyInstaller Compiler Review

Compiler Script (`build.py`):

- **Symmetric Execution**: Employs PyInstaller to compile `main.py` into a single standalone directory structure (`--onedir`). This is standard for complex PyQt6 GUI applications to ensure asset loading stability.
- **Icon Branding**: Automatically generates `icon.ico` from `logo.png` if missing (via Pillow), and embeds the multi-resolution icon metadata into the compiled Windows executable.
- **Dependency Registry**: Core libraries (`cryptography`, `psutil`, `pyqtgraph`) and optional client APIs (`routeros_api`) are correctly registered under PyInstaller hidden imports.

2. Output Packaging Issues

Our file audit of the build system highlighted a critical terminology mismatch in the zipping step:

- **Current Code Behavior (`build.py` L127-133)**:
- `# 2. Prepare Free Version`
- `prepare_dist_folder("CafePulse_Free", is_pro=False) # Wait, it passes 'CafePulse_Free' bu`
- `Wait! Let's check exports/ folder contents again:`
    - We found:
        - `exports/CafePulse_Basic.zip`
        - `exports/CafePulse_Pro.zip`
    - This is because the previous compilation runs were executed using an older version of the script, or the folders were generated with legacy names.
    - In the current `build.py` main block:
    - `128:     prepare_dist_folder("CafePulse_Free", is_pro=False)`
    - `129:     create_zip("CafePulse_Free")`
    - `130:`
    - `131:     # 3. Prepare Professional Version`
    - `132:     prepare_dist_folder("CafePulse_Professional", is_pro=True)`
    - `133:     create_zip("CafePulse_Professional")`
    - `So running the current build.py will correctly output CafePulse_Free.zip and CafePulse_`

3. Technical Recommendations

1. **Purge Legacy Zips**: Propose deleting `exports/CafePulse_Basic.zip` and `exports/CafePulse_Pro.zip` from the directory to keep only final synchronized distributions.
2. **Post-Build Validation**: Introduce checksum verification (SHA-256 generation) in `build.py` for compiled zips to allow users to verify download integrity on the website.
3. **Inno Setup Integration**: Configure `build.py` to trigger the Inno Setup CLI compiler automatically if the Windows OS environment has Inno Setup installed.

CafePulse Commercial Readiness Audit Report

This report evaluates the status of buy buttons, download links, payment gateways, licensing engines, and promotional program workflows in the CafePulse web and desktop ecosystem.

1. Gateway and Payment Integration Audit

- **Midtrans / Xendit / Stripe / PayPal**: **NOT INTEGRATED**.
- **Billing Cart/Checkout Views**: **NOT IMPLEMENTED**.
- **Purchase Button Behavior (`pricing.html`)**:
  - "Purchase Professional License" buttons redirect users locally to `./contact.html`.
  - **Status**: Pure placeholder actions.
- **Payment Pipeline Strategy**:
  - Integrating automated merchant APIs introduces monthly fees, server databases, and compliance overhead.
  - To preserve the **Local-First, Offline-First** philosophy of CafePulse, the system will use a **Manual Offline Activation Pipeline**:
    1. The customer sends a purchase request via `contact.html` or direct email to `cafepulse.network@gmail.com`.
    2. The developer replies with bank transfer details or a static QRIS payment code.
    3. Once payment is confirmed manually, the developer copies a commercial key from `100_PREGENERATED_COMMERCIAL_LICENSES.md` (or generates a custom serial using the offline generator tool) and emails it to the customer.
    4. The customer inputs their owner name and license serial key into the CafePulse desktop UI for local, offline validation.

2. Download Buttons Status

- **Live Binaries Presence**:
  - No compiled executable installer files (`.exe`, `.msi`, `.AppImage`) exist in the repository or release channels.
- **Download Page Elements (`download.html`)**:
  - The main buttons "Download for Windows (Installer)", "Download Portable (ZIP)", and "Download for Linux" are powered by dynamic API fetches in `js/main.js` which query the GitHub Releases endpoint (`releases/latest`).
  - If no releases are found or the API request fails, it redirects users to `https://github.com/yubelki/cafepulse/releases` (which is incorrect and points to the old repository URL namespace).
  - **Status**: **NOT READY**. Placeholders only.

3. Desktop Licensing Engine

- **Script Location**: `core/licensing/licensing_manager.py`
- **Cryptographic Foundation**: Extracted key signatures are verified offline using asymmetric decryption algorithms.
- **HWID Anchoring**: Licenses are tied directly to the motherboard UUID or primary CPU hardware ID to enforce the *1 License = 1 PC* policy.
- **Serial Key Database**: `docs/100_PREGENERATED_COMMERCIAL_LICENSES.md` contains 100 pre-generated key sequences mapped to the "Professional Edition" and updated with "5-Year Update

Entitlement" tags.

- **Verification Status**: **FULLY READY**. The decryption algorithm validates these keys offline in testing environments without requiring external network connectivity.

4. Promotional Programs (Founder & Beta Workspaces)

A. Founder Program (`founder.html`)

- **Objective**: Attract the first 100 users with a discounted rate (Rp 399.000 instead of Rp 499.000).
- **CTA Button Target**: Points directly to `mailto:cafepulse.network@gmail.com?subject=Founder%20Program%20Inquiry`.
- **Verification Status**: **READY (MANUAL ONLY)**. The link correctly triggers default desktop mail clients.

B. Beta Tester Program (`beta.html`)

- **Objective**: Recruits active network engineers and RT/RW Net administrators to test early builds.
- **CTA Button Target**: Points directly to `mailto:cafepulse.network@gmail.com?subject=Beta%20Program%20Application`.
- **Verification Status**: **READY (MANUAL ONLY)**. Mail client links are fully operational.

CafePulse Payment System Audit Report

This report audits the website's commercial transactions, checkout logic, and payment gateway readiness.

1. Automated Checkout Audit

We audited the entire codebase to inspect if any automated billing hooks, merchant APIs, or secure transaction scripts are active:

- **Payment Gateway (Midtrans, Xendit, Stripe, etc.)**: **NOT INTEGRATED**.
- **Cart or Checkout Page**: **NOT PRESENT**.
- **Pricing Purchase Actions**: The "Purchase License" button on `pricing.html` redirects directly to `contact.html`.
- **Transaction Status**: **No automated transaction engine exists.**

2. Inventory of Commercial Action Links

Page	Action Element	Link Target	Transaction Status
`pricing.html`	"Purchase License" Button	`./contact.html`	`MANUAL / INQUIRY ONLY`
`founder.html`	"Claim Founder Spot" Button	`mailto:cafepulse.network@gmail.com`	`MANUAL / EMAIL INQUIRY`
`beta.html`	"Apply as Contributor" Button	`mailto:cafepulse.network@gmail.com`	`MANUAL / EMAIL INQUIRY`

3. Recommended Commercial Launch Strategy

Integrating automated APIs (QRIS, credit cards) introduces monthly merchant maintenance costs, compliance burdens, and SaaS database complexities that deviate from the **Local-First, Offline-First** philosophy of CafePulse.

Propose: "Manual Offline Activation Pipeline"

1. **Purchase Request**: The buyer submits a form on `contact.html` or sends an email to `cafepulse.network@gmail.com` detailing their name and desired license count.
2. **Manual Billing**: The developer replies with a QRIS static payment code or bank transfer details.
3. **Serial Generation**: Once payment is verified, the developer copies a pre-generated commercial key from `100_PREGENERATED_COMMERCIAL_LICENSES.md` (or generates a new one using `tools/license_generator/generator.py` for the user's name) and emails it to the buyer.
4. **Offline Activation**: The buyer inputs their owner name and serial key into the CafePulse client interface. The client decrypts and verifies the key offline without hitting any licensing server.

This approach has **zero operating overhead**, requires **no database hosting**, is **100% offline-friendly**, and is fully ready for launch immediately.

CafePulse Website Reality Audit Report

This report evaluates the live structure, SEO configurations, sharing card metadata, path formatting, download references, and contact systems of the CafePulse website.

1. Domain and Base URL Verification

- **Official Repository URL**: `https://youbellkey.github.io/cafepulse-site/`
- **Finding**: The older `website/site_config.json` was set to the wrong namespace. This has been updated to the official repo URL, and the site compiler `website/config_site.py` was run to propagate this domain.
- **Status**: Mapped correctly. All canonical links, Open Graph properties, Twitter Card parameters, and Sitemap/Robots linkage point to the correct URL.

2. SEO & Sharing Metadata Verification

We audited all 9 HTML pages in the website root (`index.html`, `product.html`, `pricing.html`, `founder.html`, `beta.html`, `documentation.html`, `download.html`, `about.html`, `contact.html`):

- **Page Titles & Descriptions**: Each page has a unique `<title>` and `<meta name="description">` tailored to its contents.
- **Canonical URLs**: Configured on each page to prevent duplicate indexing penalties on search engines.
- **Open Graph / Twitter Cards**: All pages bundle Facebook Open Graph (`og:url`, `og:title`, `og:image`, `og:type`) and Twitter Card (`twitter:card`, `twitter:url`, `twitter:image`) metadata headers, pointing to the unified preview banner: `https://youbellkey.github.io/cafepulse-site/assets/og_preview.png`.

3. Site Map and Robots Settings

- **robots.txt**: Fully verified. Contains:
- `User-agent: *`
- `Allow: /`
- `Sitemap: https://youbellkey.github.io/cafepulse-site/sitemap.xml`
- **`sitemap.xml`**: Properly structured under standard sitemaps schemas. Mapped all 9 live pages with co

4. Download and Checkout Link Verification

- **Download Page (`download.html`)**:
  - Action buttons ("Download for Windows") trigger dynamic queries to the GitHub Releases API.
  - **Fallback Issue**: If the API fetch fails, the script redirects users to `https://github.com/yubelki/cafepulse/releases`. This is the incorrect namespace (pointing to `yubelki` instead of `youbellkey`).
  - **Fix Action**: The fallback link must be updated in `js/main.js` to target `https://github.com/youbellkey/cafepulse-site/releases`.
- **Purchase Button (`pricing.html`)**:
  - Redirects users to `./contact.html`. No payment gateways or carts exist.

5. Contact System & Mailto Validation

- **Mailto Link**: Points to `mailto:cafepulse.network@gmail.com`.
- **Behavior Analysis**: Clicking a `mailto` link will trigger the default system email client. However, on

machines with no email client configured, this action fails silently (leaving users on a blank browser page).

- **Technical Recommendation**:
  - Implement a "Copy Email" script using a clipboard copy action:
  - `function copyEmailToClipboard() {`
  - `  navigator.clipboard.writeText("cafepulse.network@gmail.com");`
  - `  // Trigger a toast notification: "Email copied to clipboard!"`
  - `}`
  - `Place a clickable "Copy Email Address" button next to the mailto link on the contact.html,`

6. Mobile Layout & Responsiveness

- **Responsive Styling**: Stylesheets (`css/responsive.css`) utilize CSS media queries to resize grids, shrink hero banners, adapt tables into vertical card widgets, and collapse navigation items into a hamburger menu layout on viewport sizes `< 768px`.
- **Assets Portability**: All links use relative path formats (`./`). The site loads and routes perfectly on mobile devices, tablets, and local folder previews (`file:///`).

CafePulse Download System Audit Report

This report evaluates the status of application download links, available compiled binaries, and distribution packages on the CafePulse website.

1. Binary Assets Presence Audit

We audited the repository file system and release outputs to check if installer packages exist:

- **Windows Installer (`.exe`/`.msi`)**: **NOT PRESENT**. No executable setup files have been built yet.
- **Linux Bundle (`.AppImage`/`.deb`)**: **NOT PRESENT**.
- **Zipped Distributions**: We located two files inside the `exports/` folder:
  - `exports/CafePulse_Basic.zip` (80.5 MB)
  - `exports/CafePulse_Pro.zip` (80.5 MB)
  - *Audit Finding*: These are raw PyInstaller output folders compressed into ZIP format. They still use the legacy naming conventions (`Basic` and `Pro` instead of `Free` and `Professional`).

2. Release & Download Readiness Status

Target	Deployment Status	Details
Free Edition Download	`NOT READY`	Download page targets placeholder links. Zipped source contains "Basic" branding.
Professional Edition Download	`NOT READY`	Requires serial key activation. No setup binary exists yet.

3. Website Download References Inventory

The following pages claim or link to software downloads:

1. `download.html`:
   - *Elements*: "Download for Windows" (primary EXE), "Download Portable ZIP", "Download for Linux (AppImage)".
   - *Status*: Placed as dynamic placeholders. `main.js` attempts to fetch download links from the GitHub Releases API (`releases/latest`). If the API fetch fails or no release exists, they fall back to the placeholder: `https://github.com/yubelki/cafepulse/releases`.
2. `index.html`:
   - *Elements*: "Download Free Edition" primary CTA button.
   - *Status*: Targets `./download.html`.
3. `404.html`:
   - *Elements*: "Download" button in error actions.
   - *Status*: Targets `./download.html`.
4. `founder.html` & `beta.html`:
   - *Elements*: References to downloading the software.
   - *Status*: Informational text leading users to `./download.html`.

4. Recommendations for Production

1. **Add Beta Disclaimer**: In `download.html`, add a clear informational badge:
    - "*CafePulse Version 1.0 is currently in pre-launch testing. Installers will become available immediately upon the start of the Founder and Beta programs.*"
2. **Update Zipping Scripts**: Patch `build.py` to output distributions named `CafePulse_Free.zip` and `CafePulse_Professional.zip` instead of the old Basic/Pro tags.
3. **Draft Release Pipeline**: Maintain download links targeting the GitHub Releases API structure so that uploading the binaries automatically populates the website's download button tags.

CafePulse Email & Contact Audit Report

This report audits the communication links, form submission logic, and mail client integration behaviors on the CafePulse website.

1. Mailto Link Integration Audit

We verified all instances of email links across the website:

- **Official Email Address**: `cafepulse.network@gmail.com`
- **HTML Markup**: `<a href="mailto:cafepulse.network@gmail.com">cafepulse.network@gmail.com</a>`

Usability Test Findings:

1. **With Native Mail Client**: On systems with a default email client configured (e.g., Microsoft Outlook, Windows Mail, Apple Mail), clicking the link immediately triggers the OS to launch the client and open a draft compose window addressed to `cafepulse.network@gmail.com`.
2. **Without Native Mail Client**: On systems without a configured client (common on Windows 10/11 machines where users only access web-based Gmail or Yahoo in a browser), clicking the link either:
    - Triggers a confusing Windows prompt asking the user to select an app or configure an account.
    - Fails silently, leaving the user with the impression that the website is broken.

2. Contact Form & JavaScript Validation

- `contact.html` / `beta.html` **Forms**:
    - Form inputs are validated client-side by browser default constraints (`required` attribute, type validation).
    - **Submit Behavior**: Intercepted in `website/js/main.js`. Clicking submit calls `e.preventDefault()`, shows a static text block ("Submitting message..."), and resolves via a 1.2-second timeout to show a success state: "*Message dispatched successfully to cafepulse.network@gmail.com! We will reply within 48 business hours.*"
    - *Audit Finding*: This is a simulated backend. The message is **not** transmitted anywhere because there is no server-side form handler hooked up.

3. Technical Recommendations

To optimize user experience and ensure no inquiries are lost, we recommend the following enhancements:

A. Clipboard Copy Feature (Recommended UI addition)

Add a "Copy" button next to email text links so webmail users can grab the address instantly:

```
<span class="email-wrapper">
    <a href="mailto:cafepulse.network@gmail.com">cafepulse.network@gmail.com</a>
    <button onclick="navigator.clipboard.writeText('cafepulse.network@gmail.com'); alert('Emai
</span>
```

B. Form Backend Integration

Instead of simulating submissions, map the `action` attribute of the `<form>` tag to a static handler such as **Formspree** or **Netlify Forms**:

```
<!-- Formspree Endpoint Example -->
<form action="https://formspree.io/f/your-form-id" method="POST" id="contact-form">
```

This requires zero backend coding and guarantees that customer emails land in the developer's inbox.

CafePulse GitHub Release Audit Report

This report evaluates the readiness of the CafePulse GitHub repository structure, documentations, release tagging plans, sitemap alignments, and website deployment pipelines.

1. Repository Structure & Configuration

We audited the repository root to verify file structure cleanups:

- **Branding Integrity**: Checked and confirmed that all brand assets (`logo.png`, `logo.svg`, `icon.ico`, `splash.png`) have been cleaned of legacy terms and resides under `assets/branding/`.
- **Licensing Guidelines**: Root directory contains:
  - `README_FREE.md` (installation and execution details for Free Edition).
  - `README_PROFESSIONAL.md` (router setups and offline key information for Professional Edition).
  - `LICENSE.txt` (updated copyright notice).
- **Missing Files**: No dedicated changelog file (`CHANGELOG.md`) is active in the root folder.

2. Release & Tagging Strategy

- **Target Tag Convention**: `v1.0.0`
- **Release Branch**: `main`
- **Release Package Layout**:
  - Free Edition Release: Bundles `CafePulse_Free.zip` (portable zipped binary) and `CafePulse_Free_Setup.exe` (Inno Setup compiler package).
  - Professional Edition Release: Bundles `CafePulse_Professional.zip` and `CafePulse_Professional_Setup.exe`.
- **Changelog Formatting Plan**:
  - Every release tag description must contain:
    1. A detailed list of additions and fixes.
    2. A checklist of system requirements (Windows 10/11, Python 3.12+ for source execution).
    3. Clear instructions on offline HWID serial activation.
    4. An explicit note confirming **100% Local Processing** (no user credentials or traffic packets ever leave their local machines).

3. GitHub Pages Integration

- **Official Pages Namespace**: `https://youbellkey.github.io/cafepulse-site/`
- **Subdirectory Pathing**: The website layout references resource files (CSS, JS, images, sitemap) relatively starting with `./`. This ensures the site displays perfectly inside a project subdirectory (`/cafepulse-site/`) without absolute mapping errors.
- **Domain Redirection Error Warning**:
  - **Finding**: `docs/website/` subfolders still contain legacy audit reports (`github_pages_audit.md`, `seo_audit.md`, `open_graph_report.md`, `sitemap_report.md`, `robots_report.md`) that hardcode the wrong URL `https://yubelki.github.io/cafepulse/`.
  - **Status**: While the active website files have been corrected, these legacy documentation files must be updated or removed during the final launch cleanup to prevent confusion.
- **NoJekyll Presence**: The `website/.nojekyll` file is present. This bypasses Jekyll compilation on GitHub Pages, ensuring asset folders starting with underscores (e.g. `__pycache__` if present or other subfolders)

are served.

CafePulse Ecosystem Final Audit Report

This report documents the final pre-release audit of the entire CafePulse repository to ensure terminology synchronization, single source of truth verification, and directory asset cleanup.

1. Directory Tree Audit

We inspected the contents of the repository directories to verify their active state and purpose:

Directory	Actual Status	Purpose / Finding
`assets/`	Active	Holds branding assets (`logo.png`, `logo.svg`, `logo_dark.png`, `logo_light.png`, `icon.ico`, `splash.png`). Verified 100% synchronized.
`docs/`	Active	Documentation root. Subfolders present: `legal/`, `business/`, `product/`, `architecture/`, `archive/`.
`website/`	Active	Deployed web root containing 9 HTML pages, `.nojekyll`, manifests, and localized assets.
`ui/`	Active	PyQt6 GUI desktop application widgets and window classes.
`core/`	Active	Core engine (database interface, licensing validation, network discovery, logging configurations).
`services/`	**Not Present**	There is no root-level `services/` directory. All background connectivity and MikroTik API interactions are handled inside the `core/` modules (e.g., `core/mikrotik/`, `core/scanner/`).
`database/`	Active	Contains migration configuration helpers and testing DB scripts.
`build/`	Active	PyInstaller temporary compilation folders.
`scratch/`	Active	Diagnostic, testing, and generation scripts (`generate_100_keys.py`, etc.).
`installer/`	**Not Present**	The installer compiler directory is not built yet, pending Phase 8's Inno Setup architecture decision.
`licenses/`	**Not Present**	There is no root `licenses/` folder. Official license agreements reside in `docs/legal/` and pregenerated serial keys reside in `docs/`.
`config/`	Active	Contains runtime settings (`settings.json`).
`README`	Active	Structured into `README_FREE.md` (Free Edition guidelines) and `README_PROFESSIONAL.md` (Professional Edition guidelines) in the workspace root.
`LICENSE`	Active	Stored as `LICENSE.txt` containing the copyright declaration updated to 2026.
`CHANGELOG`	**Not Present**	No dedicated CHANGELOG file exists in the root directory. Release change logs are currently drafted inside the user documentation blocks.

2. Legacy Terminology Validation

We performed code and documentation scans to ensure all old terminology has been removed and replaced with the locked business rules:

- **Approved Terms**: *Free Edition*, *Professional Edition*, *5-Year Update Entitlement*, *1 License = 1 PC*, *Best Effort Support*.
- **Banned Terms**: *Basic Edition*, *Pro Edition*, *Lifetime Updates*, *Lifetime License*, *Enterprise/MSP*, *Subscription*, *SaaS*.

Audit Findings:

1. **Source Code**: Fully clean. UI labels, sidebar items, version headers, and database methods strictly use `Free Edition` and `Professional Edition` (with no Enterprise or subscription modules).
2. **Legal and EULA Agreements**: Synchronized. `eula.md` and `license_agreement.md` strictly define the **1 PC activation lock** and **5-Year Update Entitlement**.
3. **Manual Structure**: Located and resolved two legacy occurrences of `Basic vs. Pro` and `MikroTik Pro` inside `docs/product/user_manual_structure.md` and its deployment copy, replacing them with `Free vs. Professional Editions` and `MikroTik Professional Observability`.
4. **Pregenerated Keys Document**: Located and resolved one legacy occurrence of `(PRO EDITION)` in `docs/100_PREGENERATED_COMMERCIAL_LICENSES.md` and its builder script, replacing it with `(PROFESSIONAL EDITION)`.
5. **Roadmap Audits**: Confirmed that `MASTER_PRODUCT_RELEASE_ROADMAP.md` is locked and final.

3. Conflict Analysis

- **support_policy.md vs EULA**: Checked. All responses are standardized to **Best Effort Support**. Outdated 48-hour SLAs have been removed.
- **Advisory Guidelines**: Checked. Complementary advisor updates are restricted to the **5-Year Update Entitlement** standard, avoiding "lifetime" liabilities.

CafePulse Screenshot System Review

This report audits the status, quality, and metadata parameters of the application screenshots used for marketing and documentation.

1. Phase 1 Captured Screens Verification

We verified the 5 Phase 1 screenshots generated inside the deployment assets folder (`website/assets/screenshots/`):

File Name	Target UI View	Resolution	Censor Verification	Status
`dashboard_overview.png`	Main Dashboard & KPI panels	1280x800	Hides WAN/DNS credentials.	**PASS**
`business_workspace.png`	Peak Hours & Revenue charts	1280x800	Simulation data used, no customer names.	**PASS**
`operations_workspace.png`	Hotspot Voucher Generator	1280x800	Fictional voucher codes shown.	**PASS**
`network_workspace.png`	DHCP Lease list & DNS Cache	1280x800	Hides real network client hostnames.	**PASS**
`license_manager.png`	Settings License activation tab	1280x800	Hides salt codes and pre-generated keys.	**PASS**

2. Screenshot Quality Guidelines Met

- **Dark Theme Consistency**: All screenshots utilize the official Cyber-Dark theme styles, maintaining visual branding alignment.
- **Constant Aspect Ratio**: Captured exactly at **1280x800** boundaries, avoiding stretching or layout shifting when loaded into HTML cards.
- **High Fidelity**: Extracted directly from the running PyQt6 engine via high-DPI pixmap grabs rather than external screen snips, avoiding window border artifacts.

3. Future Screenshot Roadmap (Phase 2 Captures)

When the next development iterations (Phase 4: Network wizard and topographies) are built, the following screenshots must be captured to update the documentation:

1. `vlan_wizard.png`: Step-by-step wizard GUI showing physical-to-virtual port bridge assignments.
2. `smart_troubleshooting.png`: Packet loss/latency health index scores dashboard.
3. `topology_view.png`: Auto-generated local network topology node mapping trees.
4. `backup_manager.png`: Automated scheduled backup templates configuration tables.